

Apply filters to SQL queries - SDR

Project description

SQL allows users to retrieve specific data from a large database efficiently by using keywords and filters to return only the relevant information. For example, as demonstrated in this lab, simple SQL queries can be used to find after hours login attempts, employee information from specific departments, or any other useful information. This is done by specifying what data we want to return using the SELECT, FROM, WHERE, and comparator operators. This allows us to efficiently monitor and retrieve data from large-scale databases which may be too complex to navigate using a GUI.

Retrieve after hours failed login attempts

In order to retrieve the after hours failed login attempts, we must first specify that we are selecting data from the log_in_attempts table. Next, we use the WHERE and AND operators to ensure that only login attempts that are returned were made after 6pm ('18:00' in SQL), and did not successfully login (success = 0, or FALSE in SQL). Below we can see that this query has indeed returned only login attempts that were made after 6pm and were unsuccessful.

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE login_time > '18:00' AND success = 0;
+-----+-----+-----+-----+-----+-----+-----+
--+
| event_id | username | login_date | login_time | country | ip_address | success |
+-----+-----+-----+-----+-----+-----+-----+
--+
| 2 | apatel | 2022-05-10 | 20:27:27 | CAN | 192.168.205.12 | 0 |
| 18 | pwashing | 2022-05-11 | 19:28:50 | US | 192.168.66.142 | 0 |
| 20 | tshah | 2022-05-12 | 18:56:36 | MEXICO | 192.168.109.50 | 0 |
| 28 | aestrada | 2022-05-09 | 19:28:12 | MEXICO | 192.168.27.57 | 0 |
| 34 | drosas | 2022-05-11 | 21:02:04 | US | 192.168.45.93 | 0 |
| 42 | cgriffin | 2022-05-09 | 23:04:05 | US | 192.168.4.157 | 0 |
| 52 | cjackson | 2022-05-10 | 22:07:07 | CAN | 192.168.58.57 | 0 |
| 69 | wjaffrey | 2022-05-11 | 19:55:15 | USA | 192.168.100.17 | 0 |
| 82 | abernard | 2022-05-12 | 23:38:46 | MEX | 192.168.234.49 | 0 |
```

Retrieve login attempts on specific dates

Similarly, we can use a slightly modified query with the OR operator to check for login attempts on multiple specific dates. The screenshot below indicates that we are again selecting data from the log_in_attempts table, where the login date was either May 9, 2022 or May 8, 2022. The OR operator dictates that rows of data from either date should be displayed in the table. As we can see from the login_date column, this query successfully returned the information we needed.

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1
15	lyamamot	2022-05-09	17:17:26	USA	192.168.183.51	0
24	arusso	2022-05-09	06:49:39	MEXICO	192.168.171.192	1
25	sbaelish	2022-05-09	07:04:02	US	192.168.33.137	1
26	apatel	2022-05-08	17:27:00	CANADA	192.168.123.105	1

Retrieve login attempts outside of Mexico

To retrieve login attempts made outside of Mexico, we use the NOT operator to specify that the table should return only rows where the country does not match the pattern 'MEX%'. The percentage sign indicates that any combination of characters after MEX are counted as part of the pattern, and should be excluded from output. The screenshot below confirms that this query has selected the appropriate information from the login attempts table, with no entries from Mexico in the 'country' column.

```

MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE NOT country LIKE 'MEX%';
+-----+-----+-----+-----+-----+-----+-----+
--+
| event_id | username | login_date | login_time | country | ip_address | success |
+-----+-----+-----+-----+-----+-----+-----+
--+
| 1 | jrafael | 2022-05-09 | 04:56:27 | CAN | 192.168.243.140 | 1 |
| 2 | apatel | 2022-05-10 | 20:27:27 | CAN | 192.168.205.12 | 0 |
| 3 | dkot | 2022-05-09 | 06:47:41 | USA | 192.168.151.162 | 1 |
| 4 | dkot | 2022-05-08 | 02:00:39 | USA | 192.168.178.71 | 0 |
| 5 | jrafael | 2022-05-11 | 03:05:59 | CANADA | 192.168.86.232 | 0 |
| 7 | eraab | 2022-05-11 | 01:45:14 | CAN | 192.168.170.243 | 1 |
| 8 | bisles | 2022-05-08 | 01:30:17 | US | 192.168.119.173 | 0 |
| 10 | jrafael | 2022-05-12 | 09:33:19 | CANADA | 192.168.228.221 | 0 |
| 11 | sgilmore | 2022-05-11 | 10:16:29 | CANADA | 192.168.140.81 | 0 |
| 12 | dkot | 2022-05-08 | 09:11:34 | USA | 192.168.100.158 | 1 |

```

Retrieve employees in Marketing

We can apply these query fundamentals even further by combining expressions with the AND operator. Here we are filtering for the marketing department and the East office building, using the AND operator to ensure that both conditions are met simultaneously. The screenshot below confirms that this SQL query correctly returned only the employees who are a part of the marketing department in the East building.

```

MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Marketing' AND office LIKE 'East%';
+-----+-----+-----+-----+-----+
| employee_id | device_id | username | department | office |
+-----+-----+-----+-----+-----+
| 1000 | a320b137c219 | elarson | Marketing | East-170 |
| 1052 | a192b174c940 | jdarosa | Marketing | East-195 |
| 1075 | x573y883z772 | fbautist | Marketing | East-267 |
| 1088 | k865l965m233 | rgosh | Marketing | East-157 |
| 1103 | NULL | randerss | Marketing | East-460 |
| 1156 | a184b775c707 | dellery | Marketing | East-417 |
| 1163 | h679i515j339 | cwilliam | Marketing | East-216 |
+-----+-----+-----+-----+-----+
7 rows in set (0.029 sec)

```

Retrieve employees in Finance or Sales

This problem demonstrates the use of the OR operator to search for employees who are in either the finance or sales departments. The OR operator returns an entry if either condition evaluates to True. The rest of the query is the same as before, and we see below that indeed only finance or sales employees and their information is returned.

```
MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Finance' OR department = 'Sales';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134
1010	k242l212m542	jlansky	Finance	South-109
1011	l748m120n401	drosas	Sales	South-292
1015	p611q262r945	jsoto	Finance	North-271
1017	r550s824t230	jclark	Finance	North-188
1018	s310t540u653	abellmas	Finance	North-403

Retrieve all employees not in IT

This example demonstrates the use of the NOT operator, which negates the following condition. In this case, we use the NOT operator to select only employees who are not in the IT department. This operator is another foundational building block for using logical arguments to specify exactly what information we want to retrieve from a database.

```
MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE NOT department = 'Information Technology';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434
1003	d394e816f943	sgilmore	Finance	South-153
1004	e218f877g788	eraab	Human Resources	South-127
1005	f551g340h864	gesparza	Human Resources	South-366
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134
1010	k242l212m542	jlansky	Finance	South-109
1011	l748m120n401	drosas	Sales	South-292

Summary

To recap the skills I practiced in this lab exercise, I have demonstrated a working familiarity with basic SQL syntax and commands to access specific data from a large database. I have practiced using keywords such as SELECT, FROM, WHERE, LIKE, OR, AND, NOT, and others to narrow my search to only what information is required. By becoming more familiar with these tools and practices, my understanding of database management and investigation has greatly increased. I expect to use SQL and its strengths often in the future as a cybersecurity professional for a multitude of purposes.